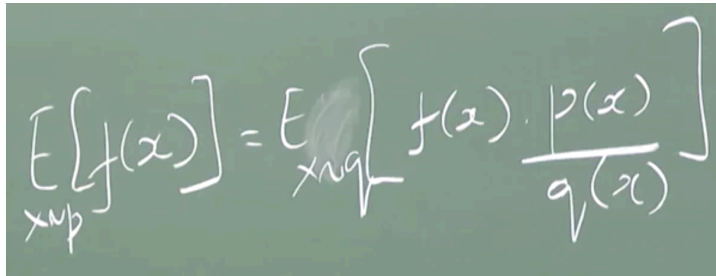


w6-L33 Off-policy MC

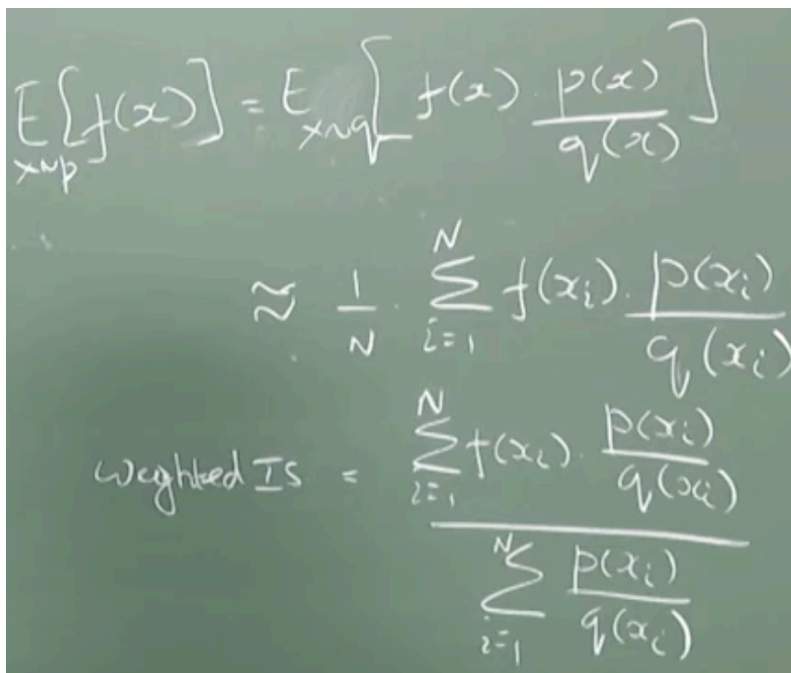
Importance sampling:

Expected values:


$$E_{x \sim p}[f(x)] = E_{x \sim q}\left[f(x) \cdot \frac{p(x)}{q(x)}\right]$$

x is sampled from p

No approx above


$$E_{x \sim p}[f(x)] = E_{x \sim q}\left[f(x) \cdot \frac{p(x)}{q(x)}\right]$$
$$\approx \frac{1}{N} \sum_{i=1}^N f(x_i) \cdot \frac{p(x_i)}{q(x_i)}$$
$$\text{Weighted IS} = \frac{\sum_{i=1}^N f(x_i) \cdot \frac{p(x_i)}{q(x_i)}}{\sum_{i=1}^N \frac{p(x_i)}{q(x_i)}}$$

Weighted or normalized importance sampling

How are we going to use in MC learning?

x_i - trajectory

$f(x_i)$ - return on trajectory

$p(x_i)$ - prob of trajectory, given policy π (interested in)

$q(x_i)$ - prob of trajectory, given actually following (μ)

π - Estimation Policy
 μ - Behavior Policy

Can leave r , they will be captured in f

$$p(x_i) = p(s_0) \pi(a_0|s_0) \cdot p(s_1|s_0, a_0) \cdot \pi(a_1|s_1) \cdot p(s_2|s_1, a_1) \cdot \dots$$
$$q(x_i) = p(s_0) \cdot \mu(a_0|s_0) \cdot p(s_1|s_0, a_0) \cdot \mu(a_1|s_1) \cdot p(s_2|s_1, a_1) \cdot \dots$$

Can we compute $p(x_i)$ - need p 's

MC's idea was we don't want to evaluate p

All I need is ratio of $p(x_i) / q(x_i)$

$$\frac{p(x_i)}{q(x_i)} = \frac{\pi(a_0|s_0) \cdot \pi(a_1|s_1) \cdot \dots}{\mu(a_0|s_0) \cdot \mu(a_1|s_1) \cdot \dots}$$
$$= \frac{\prod_{j=0}^T \pi(a_j|s_j)}{\prod_{j=0}^T \mu(a_j|s_j)} = \prod_{j=0}^T \frac{\pi(a_j|s_j)}{\mu(a_j|s_j)}$$

Pi - is used both for product and policy - disambiguate

$$E_{\pi} [f(x)] =$$

Weighted IS = value fn of some state

$$V^{\pi}(s) = \sum_{i=1}^N G_t^{(i)} \cdot \prod_{j=0}^T \frac{\pi(a_j^{(i)} | s_j^{(i)})}{\mu(a_j^{(i)} | s_j^{(i)})}$$

Super script i, coming from i-th trajectory

Return is computed from starting state s

$$V^{\pi}(s) = \frac{\sum_{i=1}^N G_t^{(i)} \cdot \prod_{j=0}^T \frac{\pi(a_j^{(i)} | s_j^{(i)})}{\mu(a_j^{(i)} | s_j^{(i)})}}{\sum_{i=1}^N \prod_{j=0}^T \frac{\pi(a_j^{(i)} | s_j^{(i)})}{\mu(a_j^{(i)} | s_j^{(i)})}}$$

This is how will do MC-policy EVALUATION

Why i want to get into Off-policy learning

- Can explore large no. of state
- Can use easier exploration policy

READ TEXTBOOK

In book, they do it incrementally,
Small Arithmetic trick - incrementally

What do you have to do for CONTROL

- How will it change for FIRST visit or EVERY VISIT
- Need action values - what is the problem?

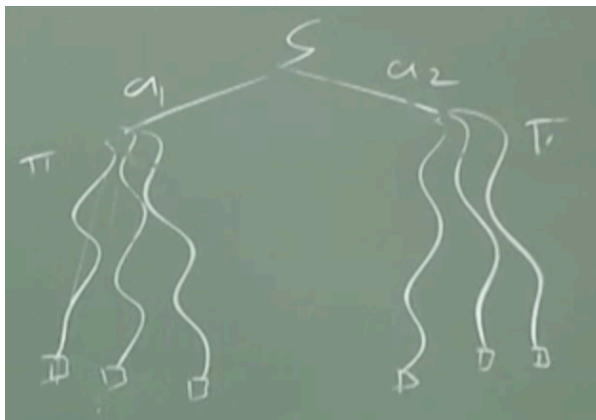
w6-L34 UCT

NOT in book - Bandit based MC planning



Techniques - ROLLOUT

- Use heuristics (equivalent to using policy)
- Run multiple trials
- Evaluate all different outcome that came from heuristics
- Pick action that gave best possible outcome



Rollout execution of policy:

NEED TO HAVE SIMULATION MODEL

Probabilistically pick action a_1 or a_2

Take average value at a_1 or a_2 - pick better average

Given any policy π - i do rollout

- At least as good as π or better
- If it can be improved (otherwise π is optimal)

This is essentially, policy evaluation of π

How many times should i run the trajectories?

Depends on - variability in the system

- Deterministic - one trajectory is enough

If variation is large in sampling

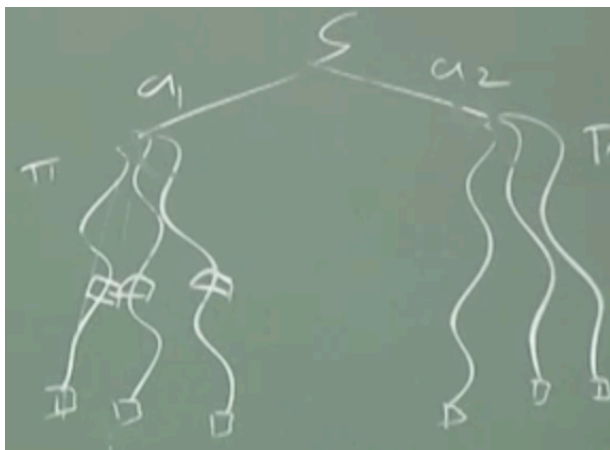
- Trajectory length - based on computation
- Longer ->

Is there a way to cut down var

- Stop somewhere in between

What is natural terminal cost

- Value function (now it is needed)



Handout from Dynamic Programming

- Xerox of few pages -> for rollout policy

Upper Confidence Tree:

- C
- Value fn
- UCB1 - bandit
- MC tree

Planning:

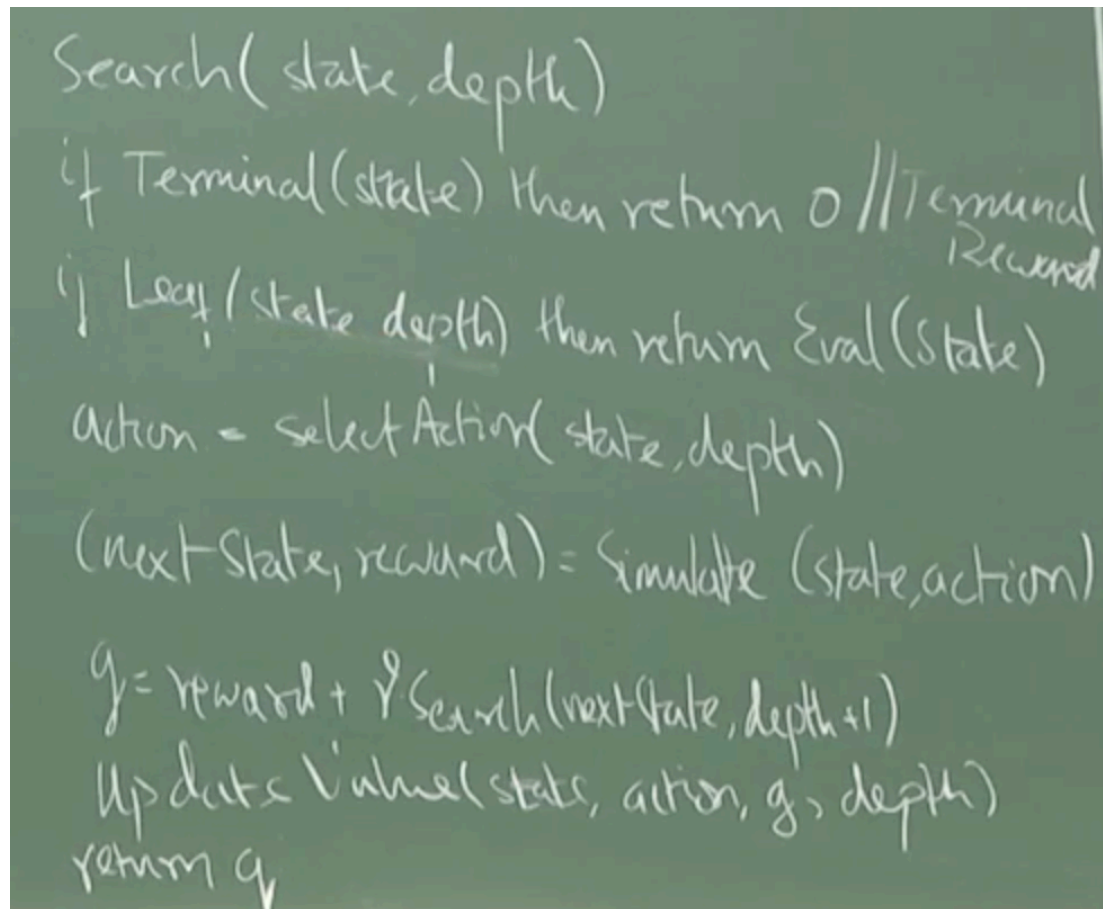
- Learning - solve MDP without knowing parameters (online)
- Planning - have some kind of model (sample or full model)

Draw samples

real - learning

Actual system - planning

MC Trees Search:



```
Search(state, depth)
if Terminal(state) then return 0 // Terminal
                                // Reward
if Leaf(state, depth) then return Eval(state)
action = selectAction(state, depth)
(nextState, reward) = simulate(state, action)
g = reward +  $\gamma$  Search(nextState, depth+1)
updateValue(state, action, g, depth)
return g
```

Search starts with state and depth
Upto some depth search
If it is terminal =0

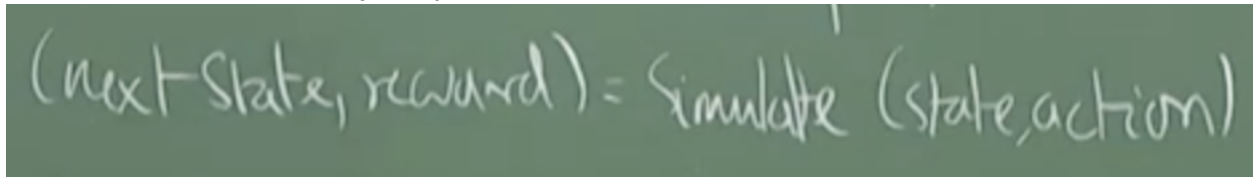
3rd line

Leaf state: will not do further processing -> return value

If not leaf, terminal state:

Select an action - how? Based on Heuristic function

Take one step - simulate trajectory



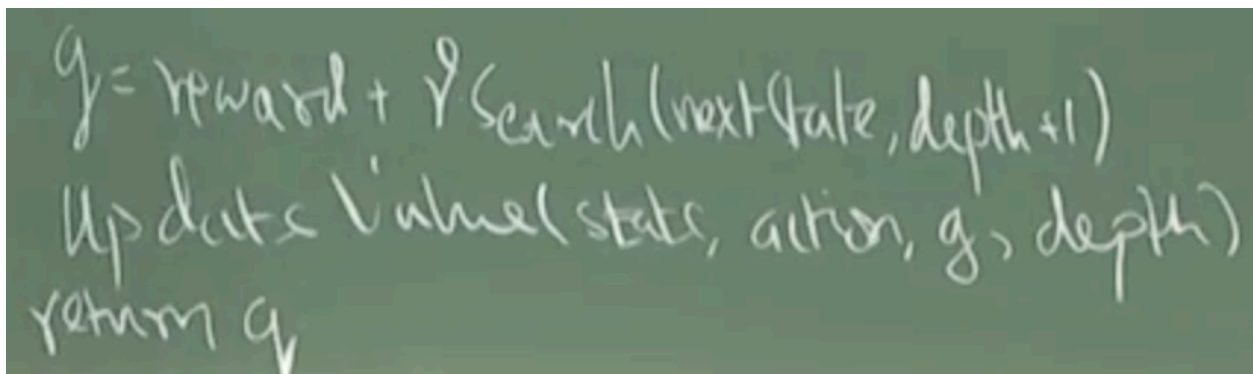
```
(nextState, reward) = simulate(state, action)
```

Need to have - simulation model

Take return, init with reward - gamma search

Once you reach a leaf, or terminal - pass (pop) reward

- Summation will have complete or adjusted return



```
g = reward + gamma * search(nextState, depth+1)
updateValue(state, action, g, depth)
return g
```

Update value of state

Depth at which stage action was taken

20min: Why is depth important?

- We are not making Markov assumption
- Closer to end - value fn will be different

When we have fixed horizon

- One step away or 100 steps away, will behave differently

Can throw away depth in last step - assuming Markov

In leaf you will need (depth of zero, can call multiple times)

- Will give one return corresponding to that
- Choose best action - based on returns

If policy has sufficient exploration - then you are fine

How will you choose best action

- Simulate each action once
- Which state gives highest value

How to recover policy - given Value function (not q function)

How will you modify it to give you q?

Basic Structure of MCTS:

MCTS:

We selection action

- have upper confidence bound type formulation
- no. of times the arm is pulled

Needed Magic Constant - some bound on it

- It is exactly like UC bandit
- Instead of having $2 \ln t$ /

you have

Instead of selecting action randomly or fixed pi

- Higher variability - larger c_p - so you want to explore more
- Determine empirically
- Plus the q value
- Go back and secure best action

What does this buy you?

- Have explore / exploit problem here
- Don't want to keep exploring all the time
- Uniform will split - waste budget
- UCT will give more focused way

UCT: To solve full-RL

- Need simulation - can come up with Rough simulator
- Robotics - very robust algorithm

Chabas - lectures -> hard to read papers

- LINEAR BANDIT
- Three classes - he took

Ans to a question: Parameterised version of value function representation

A small image of a chalkboard with the handwritten formula $\sqrt{\frac{\ln t}{n}}$ written on it.

n is no. of times action taken, $\ln t \rightarrow t$ is depth

End of ch 5:

- Recent work done by Rich Sutton
- Trying to reduce variance in MC
- When you have long trajectory - small discount factor

You don't need full product

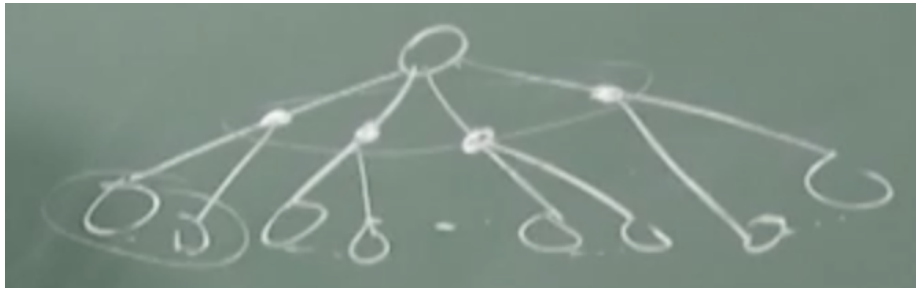
- You can get away with truncated
- How can give in principled manner
- What it means - I will not ask questions :-)

w6-L35 TD(0)

Ch6 - Book - Temporal Difference Methods (Soul of RL)

Motivation:

- Behaviour / psychology
- Computational point of view - in DP
 - Start with state s , look at all possible states



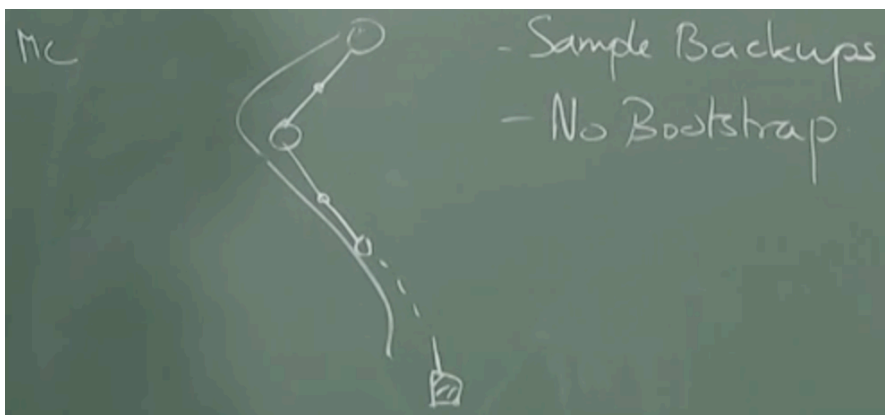
$\pi(s,a)$ actions

Sum over s'

And did backups - called Full backup using boot strapping



MC methods:



Started in a state, action, used rewards around the trajectory

Called sample backup - didn't take full outcome

- Used sample outcome
- Don't use bootstrapping

Sample backups with Bootstrap



In extreme - one sample and bootstrap

- One state simulation

Why are they called Temporal Difference?

- Used value of $t+1$ used for changing prediction at t
- Look at the difference in prediction

$$E[G_t | s_t = s]$$

G_t

In MC was interested in G_t ,

Some kind of stochastic average

$$\hat{V}_{new}(s) = \hat{V}_{old}(s) + \alpha [G_t - \hat{V}_{old}(s)]$$

This is classical stochastic averaging rule

Stochastic version of MC

$$\hat{V}_{new}(s) = \hat{V}_{old}(s) + \alpha [G_t - \hat{V}_{old}(s)]$$

"error"
"Target"
"current"

In bandit - alpha 1/n or constant
Incremental version of simple MC

Target is Expected value of G_t

Handwritten equation showing the incremental update rule for the value function $\hat{V}(s)$:

$$\hat{V}(s) = \hat{V}_{old}(s) + \alpha [G_t - \hat{V}_{old}(s)]$$

Annotations in the image:

- $\hat{V}(s)$ is labeled "new".
- $\hat{V}_{old}(s)$ is labeled "current".
- G_t is labeled "Target".
- $\hat{V}_{old}(s)$ is labeled "current".
- The term $G_t - \hat{V}_{old}(s)$ is labeled "error".

Look at target fellow closely

Handwritten derivation of the target G_t as a sum of future rewards:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

$$= R_{t+1} + \gamma G_{t+1}$$

$$\approx R_{t+1} + \gamma \hat{V}(s_{t+1})$$

Do the same trick done earlier - $v(s_{t+1})$ is true value
Can plug in above eq.

Handwritten derivation of the incremental update rule by substituting the target approximation:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

$$= R_{t+1} + \gamma G_{t+1}$$

$$\approx R_{t+1} + \gamma \hat{V}(s_{t+1})$$

$$\hat{V}_{new}(s) = \hat{V}_{old}(s) + \alpha [R_{t+1} + \gamma \hat{V}_{old}(s_{t+1}) - \hat{V}_{old}(s)]$$

15min: Remember - where $t+1$ is where is old and new is
Value fn new for state at time t
Using old value had at time $t+1$

Policy evaluation fn which uses this is called TD(0)

16min ?? Start generating trajectory

Start value fn

Create trajectory

Update value fn

At the end of

Reset the state, not value function

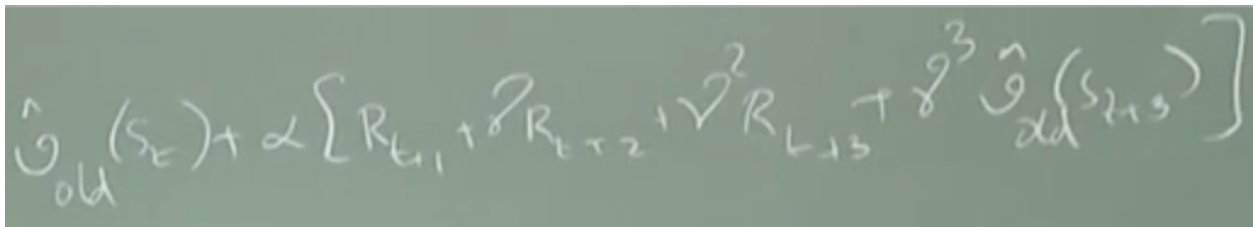
Exploration on top of basic policy (ch1 question)

- For evaluation - you should not
- If you explore - you are evaluating diff policy

TD(0) - basic algorithm

- Look at things at one step away and trying to update
- One step roll out - immediate use value fn

In Depth Limited Rollout:


$$\hat{G}_0(s_t) + \alpha [R_{t+1} + \gamma \hat{G}_0(s_{t+2}) + \gamma^2 R_{t+3} + \gamma^3 \hat{G}_0(s_{t+3})]$$

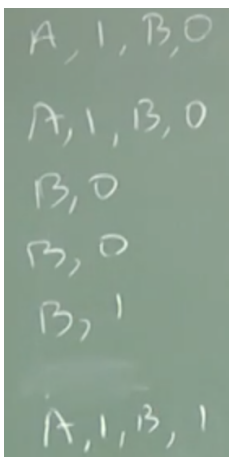
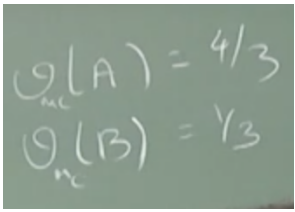
Will explain why 0 is there

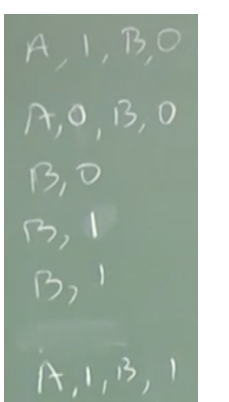
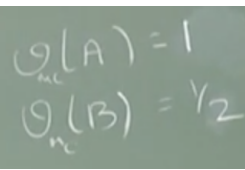
Will have family of algorithm TD(lambda)

Kalman filter - predict what next state will be

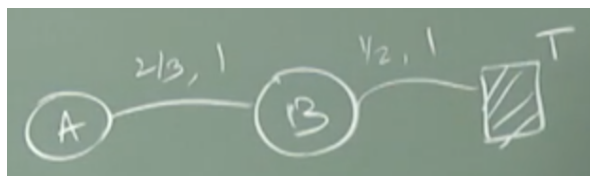
I am computing reward - from this state

Difference between TD and MC

	Dataset: seq of states and rewards you observe, $\gamma = 1$ Assume policy is fixed, is dumb  What is the value of $v(A)$, $v(B)$ Both TD and MC should give diff answers!
---	---

	MC:  $A = 1 + 2/3 = 1$, $B = 3/6 = 1/2$ TD(0): keep updating rule $v \leftarrow \text{new}$ - It computes value fn by implicitly
--	--

Expected reward for A $2/3$, prob = 1, for B $1/2$, prob 1



TD(0) value fn in this Markov chain

$$V_{TD}(B) = 1/2$$

$$V_{TD}(A) = 2/3 + \gamma V(B)$$

when $\gamma = 1$

$$V_{TD}(A) = 2/3 + V(B)$$

$$= 2/3 + 1/2$$

$$= 7/6$$

Immediate reward + value of next state

TD(0) gave different answer than MC

Which is the right answer?

- Why TD only one times of 3 gave 1
- Other times 0
- TD - assumes whole system is Markov
- From A to B to terminal -> B is considered

In MC -> don't assume structure

Which is the right answer?

- It depends on problem
- MC - return computed based on returns
- Min sq error value J_{θ} - and

TD methods

- Certainty Equivalence Model
- Model best explains data you have seen
- Try to make prediction - Certainty Equivalence Model
- Optimising different function
- **Whatever Markov model you can form based on limited data**

Infinite data - both same

Markov -> if system is NOT Markov - don't use TD

If you are using values of states - you are making Markov assumptions

- Rewards take care of non-Markovian ness

With Markov, will get depressed

w6-L36 TD(0) Control

TD(0) does evaluation

Sample backup and bootstrap

Control:

$$q_{\text{new}}(s_{t+1}, a) = q_{\text{old}}(s_t, a) + \alpha [R_{t+1} + \underbrace{q_{\text{old}}(s_{t+1}, a)}_{g(s_{t+1})_{\text{old}}} - q_{\text{old}}(s_t, a)]$$

Control:

$$q_{\text{new}}(s_{t+1}, a) = q_{\text{old}}(s_t, a) + \alpha \left[\underbrace{R_{t+1} + \underbrace{q_{\text{old}}(s_{t+1}, a)}_{g(s_{t+1})_{\text{old}}}}_{\text{Target}} - \underbrace{q_{\text{old}}(s_t, a)}_{\text{current}} \right]$$

Error

4min: TD error - error on v fn

$$\text{TD error} \equiv \delta.$$

Why using q-fn, should have taken expectation

$$\text{bootstrap} = E_{\pi_{\text{old}}} [q_v(s_{t+1}, a_{t+1})]$$

$q(s_{t+1})$

Instead of expectation, taking one sample - apply often,
Eventually will get expectation

Understand why q -old is put $\rightarrow$ unbiased estimator

Another way of doing Control:

- Instead of running several steps, take one step
- Be greedy wrt one update done
- Instead of having fixed policy
- Extreme - generalised policy

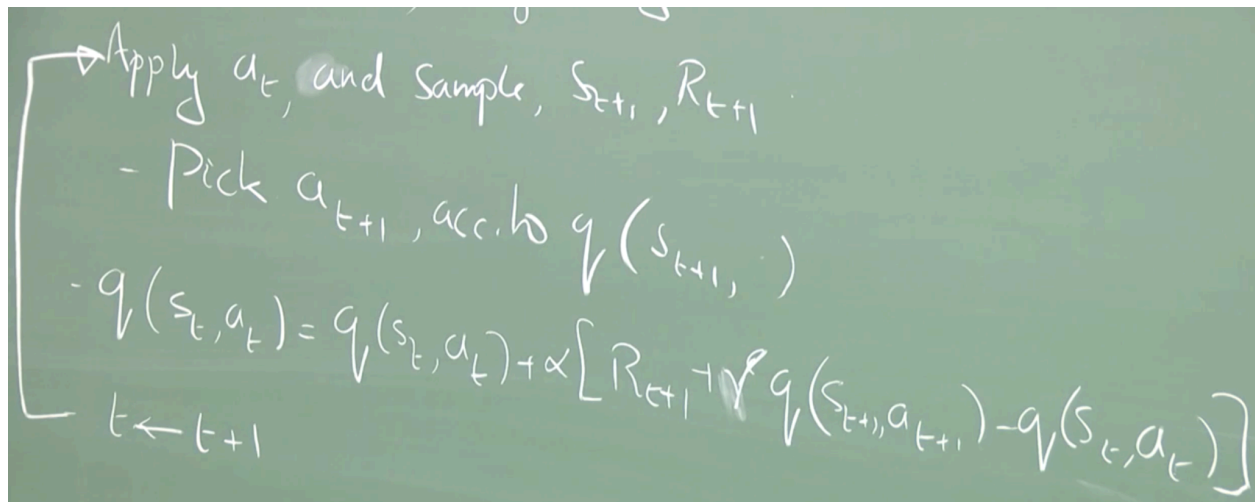
Start with arb. $q(\cdot, \cdot)$
 Pick a state s_0 , $t=0$.
 Pick action a_t in S_t , acc. to $q(s_t, \cdot)$, ϵ -greedy
 Apply a_t , and sample, s_{t+1}, R_{t+1}

According to current q -value

Being greedy is bad

- Can use softmax
- Bayesian exploration
-

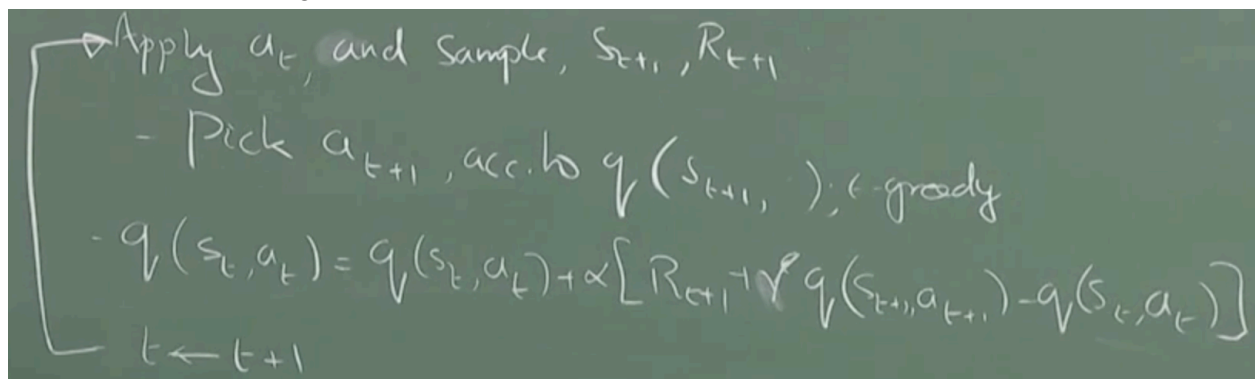
Simulate or run in simulation



Keep doing until episode terminate

Go back, until you are happy with q-fn

No explicit evaluation, greedification



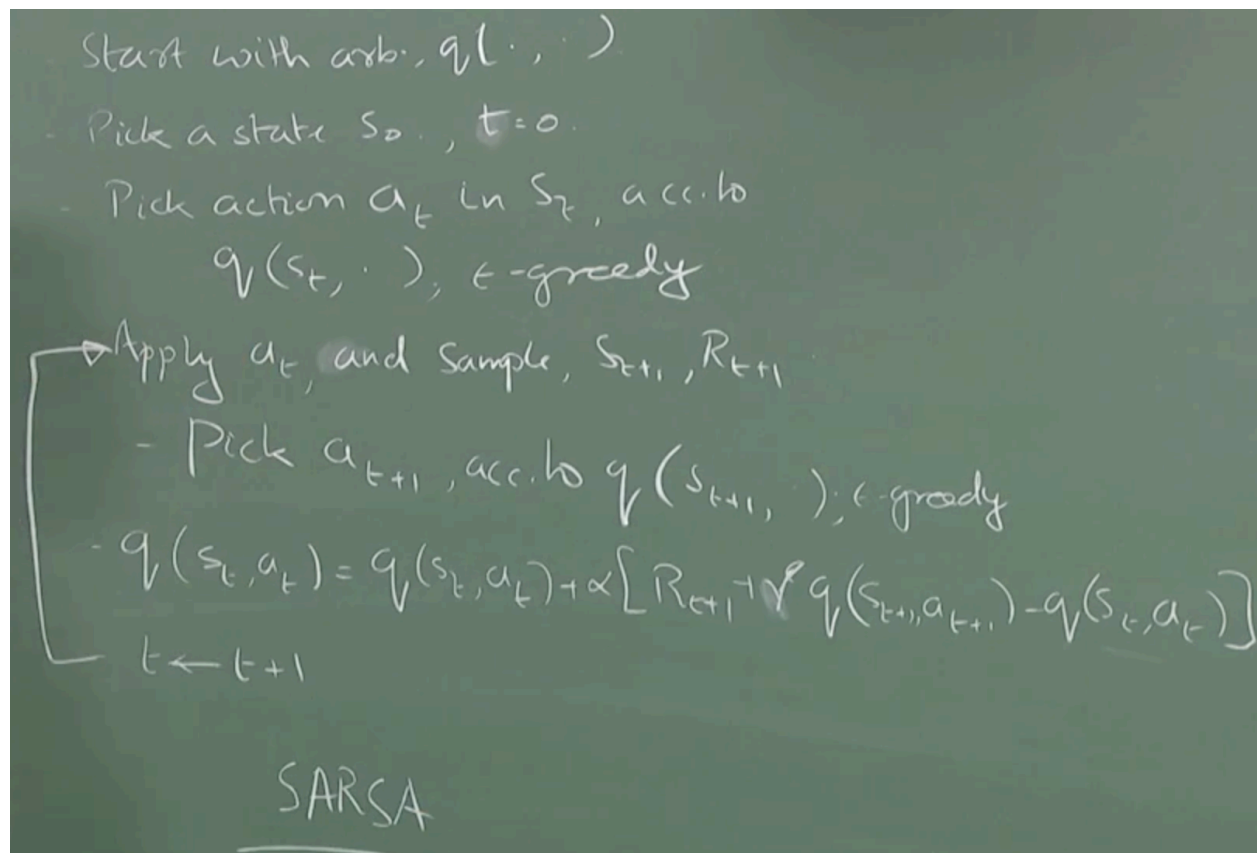
For algorithm to converge

Decrease epsilon and alpha - long time in future

- Explore

Alpha should not decay until you have explored enough

SARSA:



What is SARS-nextA-next (Mexican dish or dance!)

Dumbest possible explanation for name

$$q(s_t, a_t) = q(s_t, a_t) + \alpha [R_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t)]$$

$t \leftarrow t+1$

Rumbary and Niranjana -> Niranjana (sabbatical from England)

SARSA - On-Policy Algorithm (second most popular algorithm):

- Following epsilon greedy policy
- Also updating

I can compute this expectation - it is wrt to π , quantity I take is $q-\pi$

$$ap$$

$$E_{\pi} [q(s_{t+1}, a_{t+1})]$$

i don't have to take sample

Replace line

$$\begin{aligned} & \rightarrow \text{Apply } a_t, \text{ and sample, } s_{t+1}, R_{t+1} \\ & - \text{Pick } a_{t+1}, \text{ acc. to } q(s_{t+1}, \cdot); \epsilon\text{-greedy} \\ & * \quad q(s_t, a_t) = q(s_t, a_t) + \alpha [R_{t+1} + \gamma \sum_{a'} \pi(s_{t+1}, a') q(s_{t+1}, a') - q(s_t, a_t)] \\ & t \leftarrow t+1 \end{aligned}$$

with

$$* \quad q(s_t, a_t) = q(s_t, a_t) + \alpha [R_{t+1} + \gamma \sum_{a'} \pi(s_{t+1}, a') q(s_{t+1}, a') - q(s_t, a_t)]$$

Can do something like this

Don't have to pick action

- Forcing to take action (don't pick again)

As taking expectation, can take different action

- Because of expectation, as not evaluating action
- It is between off and on policy

When q-fn changes, policy will change

$$* \quad q(s_t, a_t) = q(s_t, a_t) + \alpha [R_{t+1} + \gamma \sum_{a'} \pi(s_{t+1}, a') q(s_{t+1}, a') - q(s_t, a_t)]$$

Expected SARSA

w6 L37 Q-Learning (30min)

Another way of Learning Control - value iteration kind of

Control

$$q_{\text{new}}(s_t, a_t) = q_{\text{old}}(s_t, a_t) + \alpha \left[\underbrace{R_{t+1} + \gamma q_{\text{old}}(s_{t+1}, a_{t+1})}_{\text{Target}} - \underbrace{q_{\text{old}}(s_t, a_t)}_{\text{current}} \right]$$

TD error $\equiv \delta$

error

That is target and current

- I am learning q function
- What does q fn tells - take action (no question of optimality)
- then behave optimally

$$q^*(s_t, a_t) = E \left[R_{t+1} + \gamma v^*(s_{t+1}) \right]$$

-
- I am really E value of $R_{t+1} + \gamma$ times whatever come after that
- No pi, which is equal to

$$q^*(s_t, a_t) = E[R_{t+1} + \gamma v^*(s_{t+1})]$$

$$= E[R_{t+1} + \gamma \max_{a'} q^*(s_{t+1}, a')]$$

Putting together the rule: - optimal value

$$q(s_t, a_t) = q(s_t, a_t) + \alpha [R_{t+1} + \gamma \max_{a'} q(s_{t+1}, a') - q(s_t, a_t)]$$

Target minus current estimate - converting to update rule

Don't have to pick up action first and then apply, move it in the loop

- Pick a_{t+1} , acc. to $q(s_{t+1}, \cdot)$; ϵ -greedy

Why is it important?

It could come back to s , then change value function, will perform different a

$$q_{\text{new}}(s_t, a_t) = q_{\text{old}}(s_t, a_t) + \alpha \left[\underbrace{R_{t+1} + \gamma q_{\text{old}}(s_{t+1}, a_{t+1})}_{\text{Target}} - \underbrace{q_{\text{old}}(s_t, a_t)}_{\text{current}} \right]$$

Error

$$\text{TD error} \equiv \delta$$

I should be updating according to sample i am taking

$$\begin{aligned}
 & - \text{Pick } a_{t+1}, \text{ acc. to } q(s_{t+1}, \cdot); \epsilon\text{-greedy} \\
 & q(s_t, a_t) = q(s_t, a_t) + \alpha [R_{t+1} + \gamma \max_{a'} q(s_{t+1}, a') - q(s_t, a_t)]
 \end{aligned}$$

Instead of picking action before hand and stick with it

Later on we will see how we will be using approximate value function

- Talked about contextual bandit, policy parameterisation
- Can use value function parameterisation
 - It become tricky - trajectory along which updating and trajectory taking (will be messy)
 - May lead to problem with convergence
- As long as we are performing - it is ok
- As epsilon and alpha go to zero -> will converge

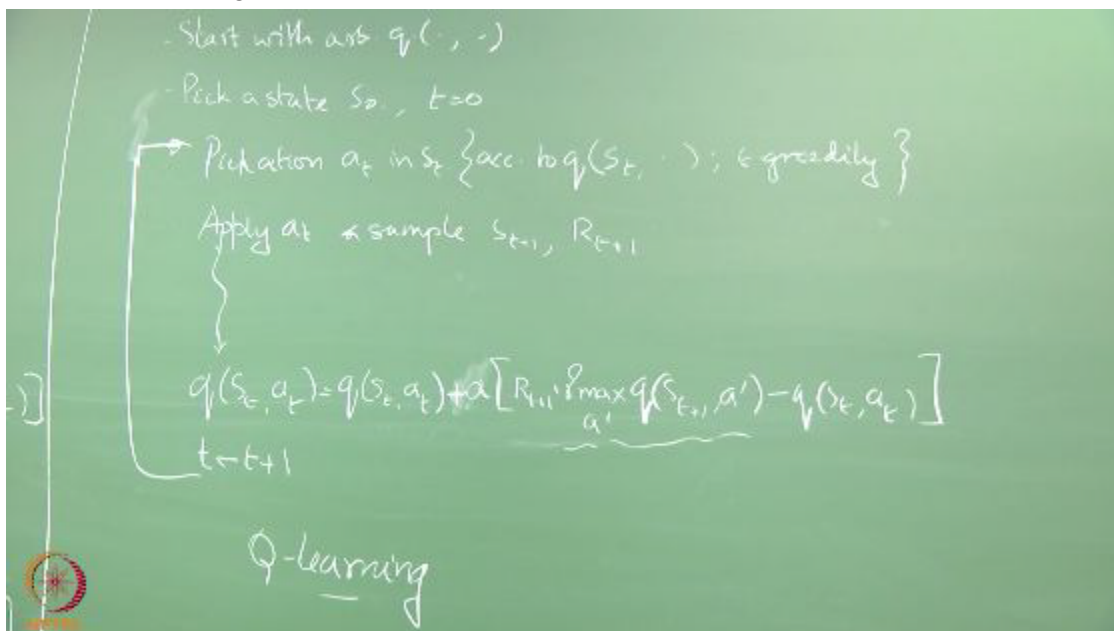
You could start with a policy

Start with arb. $q(\cdot, \cdot)$

Or make it zero - uniform random policy

- All actions equal prob

Can do same thing here:



Execution happens after you have done the updation
 Here picking action before updation is happening: using q-old

$$\begin{aligned}
 & \text{Pick } a_{t+1}, \text{ acc. to } q(s_{t+1}, \cdot); \epsilon\text{-greedy} \\
 & q(s_t, a_t) = q(s_t, a_t) + \alpha [R_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t)]
 \end{aligned}$$

Start with arb. $q(\cdot, \cdot)$

- Pick a state s_0 , $t=0$.
- Pick action a_t in S_t , acc. to $q(s_t, \cdot)$, ϵ -greedy

→ Apply a_t , and sample, s_{t+1}, R_{t+1}

- Pick a_{t+1} , acc. to $q(s_{t+1}, \cdot)$, ϵ -greedy
- * $q(s_t, a_t) = q(s_t, a_t) + \alpha [R_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t)]$
- $t \leftarrow t+1$

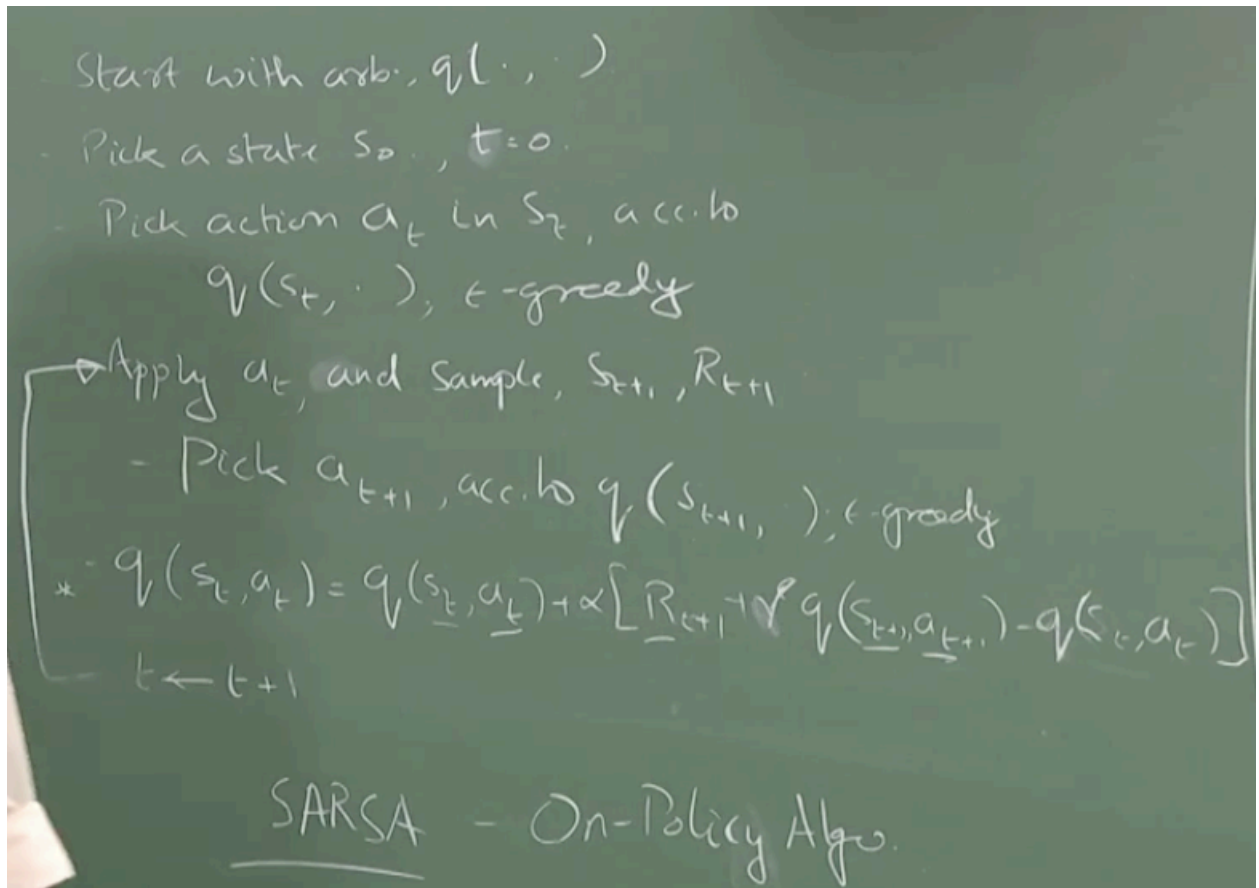
SARSA - On-Policy Algo.

Compare this algorithm with Q-learning algorithm

Decrease alpha and epsilon to converge properly

Q-learning: most popular learning algorithm (SARSA -second most)

Here using q-new for picking action:



Sum $\alpha(t) \rightarrow \text{infinity}$ - tells you are making enough update

Sum $\alpha(t)^2 < \text{infinity}$ (bounded) - you will eventually stop making updates

Decrease α to converge properly

Chronologically: TD(0) came first - Rich - 1984, showed convergence in 1988

Q-learning: Chris Watkins - 1989

SARSA - Rummery and Niranjan 1991 - RL algos came from England

What is more amazing about Q-learning:

Q-learning is amazing so what is even more amazing about Q-learning is that since I do not care what is the **sequence on which you are picking the actions** right here I am amusing an **approximation to the optimal return** right I am **not looking at the current return**.

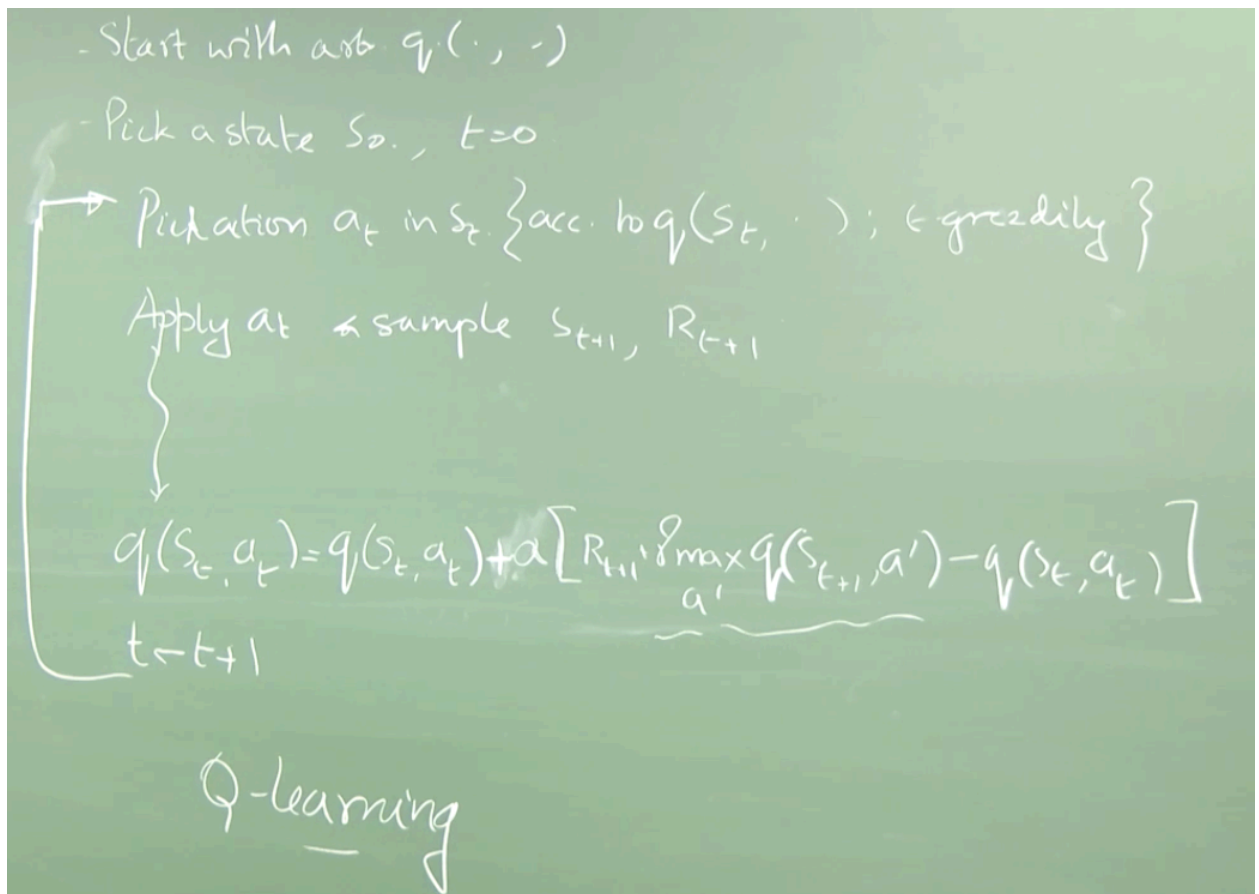
If I am looking an approximation to the optimal return so I **do not care about the trajectory** in fact in Q-learning I **do not even have to sample according to any trajectory** in fact the next action I execute **need not even come** from S_{t+1} , I can arbitrarily reset if I have a simulation model right I can arbitrarily reset to some other state right And then sample one action from there update Q-learning and then go and

take action from somewhere else, some other state update you think you can just do a **proper asynchronous generalized policy iteration** kind of an approach, I just sample from all over the place just take one step each and keep updating it.

But still it is better to update it along trajectories like we saw in the argument for RTDP right, if you updated along trajectories it is more meaningful because the values should have more likely to have changed along those trajectories right. So that is essentially why we sample according to the trajectories.

So this thing need not be an greedy sample at all right. So I can sample actions arbitrarily I can just take pick an action A_t and S_t randomly okay, I does not have to follow any rule. So what does this do it just tells me which action I am going to update next right. If I am not sampling according to trajectories it only tell me which action I am going to update next.

So why do you want to pick according to this, so that you can just like an RTDP so you can focus the updates along the most most likely paths, so what the reason you do it Σ greedy fashion is that you quickly want to converge to the most likely path that you are going to take under optimal policy and you want to do the updates along that most likely path okay. And that is why you use the Σ greedy sampling for Q-learning.



I am using approximation to return

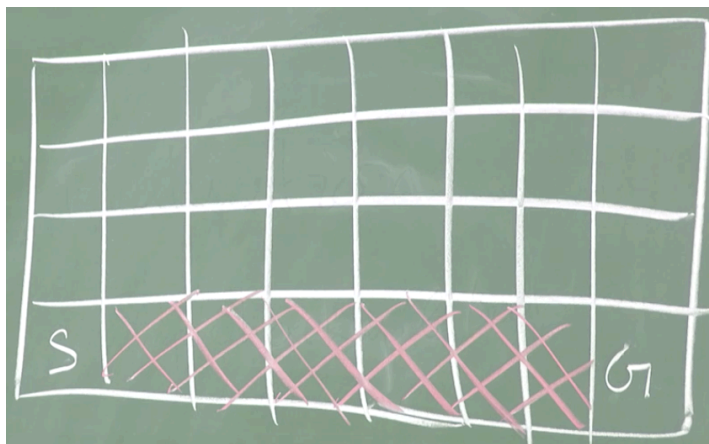
- Don't have to sample from trajectory
- I can arbitrarily reset to sample state
- Take action from some other state
- Sample from all over the place

It is better to update along trajectories

Need not be epsilon-greedy sample at all

And so what are the advantage of SARSA, SARSA you can continue to explore right and while exploring you learn something that is taking into account the exploration. So what does Q-learning do that is why I am saying you can arbitrarily with Q-learning but if you choose to behave according to the Q function you are learning, here it does not take into account the fact that you are doing exploration okay.

So why would that matter?



red represent cliff

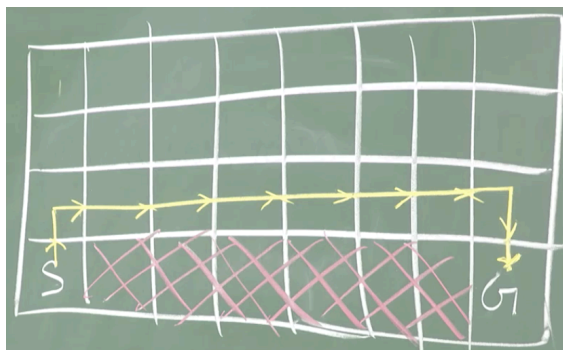
Agent wanders over cliff get -100, agent will die - reincarnate

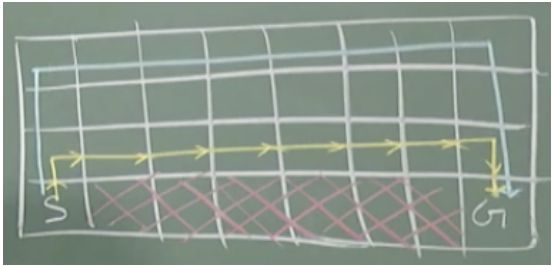
Otherwise every time step reward of -1, $\gamma = 1$ (no gamma, doesn't mean $\gamma = 0$)

Epsilon greedy selection = 0.1

What do you think Q-learning will do, what will SARSA do?

This is the optimal policy:



Will Q-learning find this optimal policy: - yes	Will SARSA find this optimal policy: - No in this case i have fixe epsilon = 0.1 What will SARSA learn?  It will take safer and safer row - It top more rows, it will go up - Will stay as far away as possible - Reward is sufficiently -ve
Q-learning - Might learning optimal policy - Will kill you more often	SARSA take into account - exploration
Q-learning - Ignores exploration	It is better to learn after sufficiently learning - Being able to explore -

If you are going to continue to explore after you finish learning right it is better to learn the policy through SARSA then through Q-learning, because Q-learning assumes that eventually you are going to be executing the optimal policy right.

Q-learning - Off-Policy:

- Exploring more - arbitrary exploration - still learn optimal policy
- Estimation policy - optimal policy
- Behaviour policy - can be completely differently
- Can explore larger section of state space

If world is not changing - you can start behaving greedily

You don't want to be completely adaptable when things change

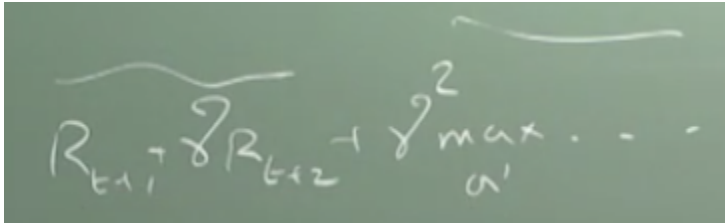
Which will be slow:

Q-learning: - Converges faster than sarsa Not sure about large state space	SARSA:
--	---------------

For large state space you will not be using vanilla Q-learning or SARSA

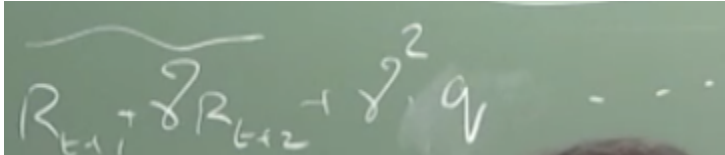
You can think of Hybrid algorithms

2 steps according to current policy then chose max

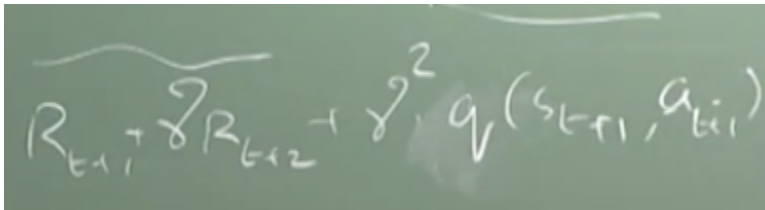

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 \max_{a'} \dots$$

First two terms - on policy

Later on - off policy


$$R_{t+1} + \gamma R_{t+2} + \gamma^2 q \dots$$

It is more common to use q function there


$$R_{t+1} + \gamma R_{t+2} + \gamma^2 q(s_{t+1}, a_{t+1})$$

Using more of actual sampling, less of boot strapping (because of gamma)

RL to play ad-hock -> in the simulator 3 step return, then the q function of rest

- For whatever reason
- Stochastic,
- Not easy to figure out what is the right look ahead

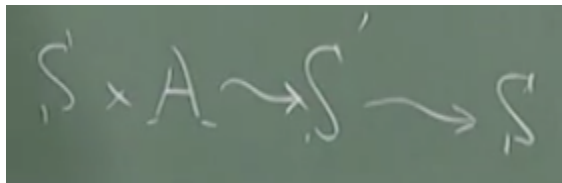
w6 L38 Afterstate (7min)

Stochasticity - nature (cows agin), opponent

You make move - no stochasticity - it will land up in square you want to go

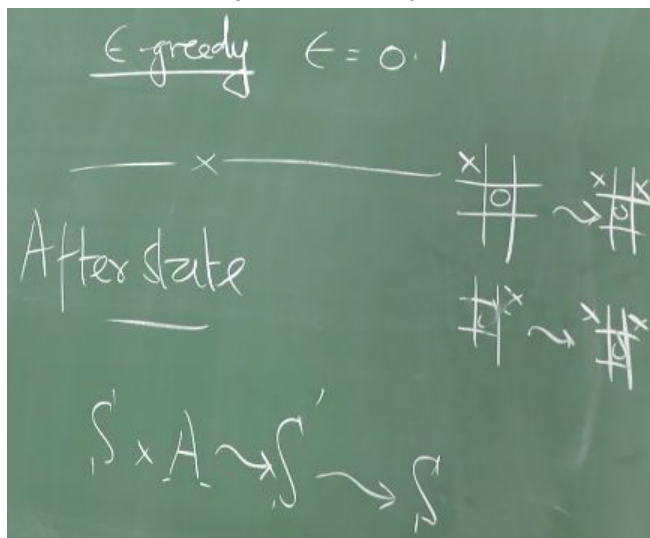
There is deterministic component and stochastic

after state: $S \times A \rightarrow S' \rightarrow S$



If i am going to represent things as after state, instead of learning q function

- Can learn v function on after state S' (over the entres)
- V fn = q fn
- Advantage - one less dimension
- Many state action pair yield same S'
- Tick-Tac-To - many different ways i could come there



- after state (2nd col is same)
- S will have equal no. (x and o) -> S' will have one more
- In tick-tac-to after states are not states
 - You will not see a state which has more xs than os
 - Could be same or different
- Since whole thing is deterministic
 - I can do look ahead - i just need to see after state after taking action
 - I can chose after

Jerry Tussaro - BackGammon